SPAM MESSAGE PREDICTION SYSTEM

Machine Learning • Natural Language Processing • Text Classification

Student: Kavyansh Bhardwaj
Institution: SIMS Delhi
Course / Drive: Artificial Intelligence and Internship Drive
Internship Organization: Naviotech
Submission Deadline: 30 October 2026

Academic project report
Prepared as a working educational prototype. The source code and supplied starter data should be tested and extended before final assessment.

1. Abstract

This project demonstrates a supervised machine-learning system that classifies a user-entered SMS or short message as spam or ham (likely legitimate). Text is converted into TF-IDF features and classified using Multinomial Naive Bayes. A Streamlit interface accepts messages and displays a prediction, confidence estimate and evaluation information. The supplied starter dataset is illustrative and must be replaced with the full UCI SMS Spam Collection for a meaningful academic evaluation.

2. Problem statement

Unwanted messages may contain misleading offers, suspicious links or fraudulent requests. Manually screening every message is inconvenient. The project explores how supervised text classification can assist by flagging potentially unwanted content. It does not inspect email headers, attachments, sender reputation or live URLs, and it cannot guarantee that a message is safe.

3. Objectives

• Prepare labelled message data for training and testing.

• Convert text into numerical features using TF-IDF.

• Train a Multinomial Naive Bayes classifier.

• Evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.

• Provide a simple interface for message prediction.

4. System architecture and workflow

User message → text input → TF-IDF vectorizer → trained Multinomial Naive Bayes classifier → spam/ham prediction → confidence and evaluation display.

5. Technology stack

Technology	Role
Python	Core programming language
Pandas	Load and prepare labelled message data
Scikit-learn	Train model, vectorize text and calculate metrics
TF-IDF	Represent message terms as weighted numerical features
Multinomial Naive Bayes	Probabilistic text classification algorithm
Streamlit	Interactive browser-based interface

6. Implementation guide

1. Install Python 3.10 or newer.

2. Open the project folder in VS Code or a terminal.

3. Create and activate a virtual environment: `python -m venv venv` then `venv\Scripts\activate` on Windows.

4. Install dependencies: `pip install -r requirements.txt`.

5. Run the app: `streamlit run app.py`.

6. Enter test messages, record predictions and capture screenshots.

7. Replace the small illustrative CSV with the full UCI dataset before reporting meaningful performance.

7. Advantages

• Fast, lightweight baseline model suitable for a classroom NLP demonstration.

• Pipeline is transparent and easy to explain in a viva.

• Provides measurable classification metrics.

• Can be extended with additional text features and alternative classifiers.

8. Disadvantages and limitations

• Small starter dataset is not representative and may produce unstable results.

• False positives and false negatives are possible.

• English SMS examples may not generalize to other languages or email formats.

• TF-IDF does not understand meaning as deeply as contextual language models.

9. Practical applications

• Initial screening of SMS or short text messages.

• Educational demonstration of NLP and supervised classification.

• Prototype support for a moderation or message-triage workflow (with human review).

10. Future scope

• Train and compare Multinomial Naive Bayes, Logistic Regression and Linear SVM.

• Use larger, multilingual and more recent labelled message datasets.

• Add URL, sender and email-header risk signals where legally and technically appropriate.

• Track false-positive rates and add a human review option.

11. Conclusion

The project demonstrates an end-to-end supervised NLP workflow, from labelled text and TF-IDF feature extraction to classification and a browser-based interface. Its current sample data is a teaching scaffold, not evidence of production accuracy. A stronger submission should use the full UCI collection, record measured test metrics, include original screenshots and discuss errors.

12. References

• Almeida, T. & Hidalgo, J. (2011). SMS Spam Collection. UCI Machine Learning Repository. DOI: 10.24432/C5CC84.

• Scikit-learn documentation: https://scikit-learn.org/.

• Streamlit documentation: https://docs.streamlit.io/.

Academic integrity and result reporting: Run the application yourself, add screenshots from your own run, record actual measurements, and acknowledge any external datasets or code used. Do not invent accuracy, user-study results, or deployment claims.